

[8장] 중앙처리장치 (CPU) - 절별 초정밀 정리

8.1 CPU 레지스터 조직 (Register Organization)

- **핵심 요약:** CPU 내부에서 데이터를 임시 저장하는 가장 빠른 고속 메모리 장치들입니다. 공통 버스(Bus) 시스템을 통해 레지스터 간에 데이터가 이동하며, 멀티플렉서(MUX)와 디코더를 활용해 제어 단어(Control Word)로 제어됩니다.
- **시험 저격 포인트:** 레지스터 개수가 N 개일 때 MUX의 선택 신호(Select Line) 비트 수는 $\log_2 N$ 이 된다는 규칙을 객관식이나 단답형 수치 문제로 자주 출제합니다.

8.2 스택 조직 (Stack Organization)

- **핵심 요약:** LIFO(Last-In, First-Out) 구조로 데이터를 관리하는 공간입니다. 레지스터 스택과 메모리의 일부를 사용하는 메모리 스택이 있습니다.
- **수식 변환과 계산 (★ 주관식 확정):**
 - **Infix (중위):** $A + B$ 형태로 사람이 쓰는 식
 - **Postfix (후위):** $A B +$ 형태로 컴파일러가 스택을 쓰기 가장 좋은 식
- **치트키:** Postfix 연산 문제를 마주하면, "숫자는 스택에 **Push**, 연산자를 만나면 2개를 **Pop** 해서 계산 후 결과를 다시 **Push** 한다"는 메커니즘을 그대로 수치에 대입해 풀면 됩니다.

8.3 명령어 형식 (Instruction Formats)

- **핵심 요약:** 명령어 내 주소(오퍼랜드) 필드의 개수에 따라 CPU의 구조와 프로그램의 길이가 달라집니다.
- **4가지 형식 비교:**
 - **3-주소:** $ADD\ R1, R2, R3 \rightarrow$ 프로그램 길이는 짧아지나 명령어 비트 수가 너무 커짐.
 - **2-주소:** $ADD\ R1, R2 \rightarrow$ 가장 일반적인 상용 구조.
 - **1-주소:** $ADD\ X \rightarrow$ 누산기(AC) 레지스터를 묵시적으로 사용하여 연산.
 - **0-주소:** $ADD \rightarrow$ 주소 필드가 없으며 스택(Stack)의 최상위 데이터들을 이용해 연산.

8.4 주소지정 방식 (Addressing Modes)

- **핵심 요약:** 명령어의 오퍼랜드 필드를 이용해 실제 데이터가 위치한 유효 주소(EA)를 찾아내는 규칙입니다.
- **핵심 방식 5가지 정의:**
 - **Immediate (즉시):** 오퍼랜드 필드에 실제 데이터 값이 들어있음 (메모리 접근 0회, 가장 빠름).
 - **Direct (직접):** 오퍼랜드가 데이터가 위치한 실제 메모리 주소($EA = A$)를 직접 가리킴.
 - **Indirect (간접):** 오퍼랜드가 가리키는 주소에 가보면 진짜 데이터의 주소($EA = M[A]$)가 들어있음 (메모리 접근 2회 필요).
 - **Relative (상대):** 프로그램 카운터 값과 더함 ($EA = PC + A$).
 - **Indexed (인덱스):** 인덱스 레지스터 값과 더함 ($EA = XR + A$).

8.5 데이터 전송 및 조작 (Data Transfer and Manipulation)

- **핵심 요약:** CPU가 수행하는 명령어 군을 기능별로 분류합니다.

- **분류:** `MOVE`, `LOAD`, `STORE` 등의 **데이터 전송 명령어**, 산술/논리/시프트 등의 **데이터 조작 명령어**, 그리고 조건 분기를 처리하는 **프로그램 제어 명령어**로 나뉩니다.

8.6 프로그램 제어 (Program Control)

- **핵심 요약:** 프로그램의 실행 흐름을 바꾸는 분기(Branch) 명령어와 서브루틴 호출을 다룹니다.
- **상태 비트 조건 (Status Bits):** 연산 결과에 따라 세팅되는 **C(Carry)**, **S(Sign)**, **Z(Zero)**, **V(Overflow)** 플래그의 상태를 보고 조건 분기 명령어가 동작하게 됩니다.

8.7 RISC vs CISC 특징

- **핵심 요약:** CPU의 명령어 집합 설계 철학에 따른 두 가지 대조적인 구조입니다.
- **CISC:** 많은 수의 복잡한 명령어, 가변 길이 명령어, 다양한 주소지정 방식, 마이크로프로그램 제어 방식.
- **RISC:** 적은 수의 단순한 명령어, 고정 길이 명령어(1클럭 처리), 대규모 레지스터 배치, 하드와이어드 제어, 파이프라인 최적화 설계.

[9장] 파이프라인 및 벡터 처리 - 절별 초정밀 정리

9.1 병렬 처리 (Parallel Processing)

- **핵심 요약:** 다수의 데이터를 동시에 처리하여 계산 속도를 향상시키는 하드웨어 기법입니다.
- **Flynn의 분류 (4가지 구조 구조):**
 - **SISD:** 단일 명령어, 단일 데이터 (전통적 구조).
 - **SIMD:** 단일 명령어, 다중 데이터 (벡터/배열 처리).
 - **MISD:** 다중 명령어, 단일 데이터 (이론적 구조).
 - **MIMD:** 다중 명령어, 다중 데이터 (멀티프로세서 구조).

9.2 파이프라인 (Pipelining)

- **핵심 요약:** 하나의 공정을 여러 개의 독립된 서브 단계(Segment)로 분할하여, 서로 다른 데이터를 동시 병렬로 연속 처리하는 선형 구조입니다.
- **속도 향상비(\$\$\$) 공식 (★ 무조건 계산 출제):**
$$$$$ = \frac{n \cdot t_n}{(k + n - 1) \cdot t_p} $$$
 - k : 파이프라인의 단계(세그먼트) 수
 - n : 처리할 전체 태스크(데이터) 개수
 - t_n : 비파이프라인 시스템에서 걸리는 총 시간
 - t_p : 파이프라인 시스템의 1클럭 주기 시간
 - **치트키:** 데이터 개수 n 이 무한대로 커지면 최대 속도 향상비는 단계 수인 k 배에 수렴합니다.$$

9.3 산술 파이프라인 (Arithmetic Pipeline)

- **핵심 요약:** 부동소수점 산술 연산처럼 복잡한 연산 과정을 여러 단계로 쪼개어 파이프라인화하는 기법입니다.
- **부동소수점 가산 4단계 순서 (단답형 대비):**

1. 지수 비교 (동기화) $\rightarrow$ 2. 가수 정렬 $\rightarrow$ 3. 가수 덧셈/뺄셈 $\rightarrow$ 4. 결과 정

9.4 명령어 파이프라인 (Instruction Pipeline)

- **핵심 요약:** CPU가 명령어를 읽고 실행하는 과정을 파이프라인 처리하는 기법입니다.
- **일반적인 4단계 구성:** FI(명령어 인출) $\rightarrow$ DA(명령어 해독 및 유효주소 계산) $\rightarrow$ FO(오퍼랜드 인출) $\rightarrow$ EX(명령어 실행)

9.5 파이프라인 해저드 (Pipeline Hazards)

- **핵심 요약:** 파이프라인의 연속적인 흐름이 깨지고 멈추게 되는 장애 현상입니다. (★ 주관식 서술 단골)
- **3대 해저드 원인과 해결책:**
 - **구조적 해저드:** 여러 단계가 동일한 물리적 자원(메모리)에 동시 접근 시 발생 $\rightarrow$ 해결: 명령용/데이터용 메모리 분리(하버드 구조).
 - **데이터 해저드:** 앞 명령어의 연산 결과가 레지스터에 기록되기 전에 뒤 명령어가 이를 읽으려 할 때 발생 $\rightarrow$ 해결: 데이터 포워딩(우회 경로 하드웨어 구축), 지연 삽입.
 - **제어 해저드:** 분기 명령어(Branch, Jump)로 인해 다음에 실행할 주소가 바뀌어 미리 채워둔 파이프라인이 깨질 때 발생 $\rightarrow$ 해결: 분기 예측, 지연 분기(NOP 삽입).

9.6 벡터 처리 (Vector Processing)

- **핵심 요약:** 과학 계산용 루프 코드를 고속 처리하기 위해, 배열 및 벡터 연산을 전용 하드웨어 파이프라인으로 단 한 번의 명령어로 처리하는 기술입니다.

9.7 배열 프로세서 (Array Processor)

- **핵심 요약:** 여러 개의 프로세싱 엘리먼트(PE)를 동기화하여 배치하고, 하나의 제어 유닛(CU)을 통해 동일한 명령어를 대규모 데이터 배열에 동시에 수행하는 동종 병렬 컴퓨터 구조(SIMD 구조)입니다.

[12장] 메모리 구조 - 절별 초정밀 정리

12.1 메모리 계층 (Memory Hierarchy)

- **핵심 요약:** 용량, 속도, 가격의 Trade-off (상충 관계)를 해결하기 위해 레지스터 $\rightarrow$ 캐시 $\rightarrow$ 주기억장치 $\rightarrow$ 보조기억장치 순서로 계층을 쌓은 시스템입니다.
- **시험 저격 포인트:**
 - **목적:** "전체 시스템의 가격을 최소화하면서 최고의 평균 접근 속도를 달성하기 위해"
 - **접근 시간 비율:** 캐시 : 주기억장치 : 보조기억장치 = \$1 : 10 : 1000\$
 - **블록 단위:** 캐시는 \$1 \sim 16\$ 워드, 보조기억장치는 \$256 \sim 2048\$ 워드.

12.2 주기억장치 (Main Memory)

- **핵심 요약:** CPU가 런타임에 직접 접근하여 프로그램 명령어와 데이터를 읽고 쓰는 공간입니다.
- **SRAM vs DRAM 핵심 비교:**
 - **SRAM:** 플립플롭 구조, 재충전 불필요, 속도 매우 빠름, 고가 $\rightarrow$ 캐시 메모리에 사용.
 - **DRAM:** 콘덴서 구조, 주기적인 재충전(Refresh) 회로 필요, 대용량화 유리, 저가 $\rightarrow$ 주기억장치

(RAM)에 사용.

- **부트스트랩 로더 (Bootstrap Loader):** 전원이 켜질 때 초기화를 수행하고 OS를 보조기억장치에서 주기억장치로 적재하는 ROM의 초기화 프로그램입니다.

12.3 보조기억장치 (Auxiliary Memory)

- **핵심 요약:** 대용량 데이터를 전원 없이도 영구 보존하는 비휘발성 저장장치(디스크 등)입니다.
- **접근 시간 구성:** 원하는 트랙을 찾는 시크 타임(Seek Time, 탐색 시간)과 트랙 내에서 해당 섹터가 회전해 올 때까지 기다리는 로테이션 딜레이(회전 지연 시간)의 합으로 결정됩니다.

12.4 어소시어티브 메모리 (Associative Memory)

- **핵심 요약:** 주소(Address)가 아니라 데이터의 내용(Content) 자체를 비교하여 초고속 탐색을 수행하는 하드웨어 메모리(CAM)입니다.
- **3대 하드웨어 레지스터 (빈칸 채우기 대비):**
 1. **Argument Register (\$A\$):** 찾고자 하는 데이터 단어를 저장.
 2. **Key Register (\$K\$):** 마스크(Mask) 비트를 지정하여 원하는 특정 비트 구역만 선별 비교.
 3. **Match Register (\$M\$):** 일치하는 데이터가 발견된 방의 해당 비트를 1로 표시.

12.5 캐시 메모리 (Cache Memory)

- **핵심 요약:** CPU와 느린 RAM 사이의 속도 버퍼 역할을 하며, **참조의 국한성(시간/공간 국한성)** 이론 덕분에 높은 효율을 냅니다.
- **캐시 매핑 3가지 주소 구조 (★ 계산 및 구조도 주관식 1순위):**
 - **직접 매핑:** 주소를 [Tag | Index]로 분할. 지정된 인덱스 방으로만 들어가야 하므로 구조가 단순하고 비용이 싸지만 주소 끝자리가 같을 때 발생하는 자원 충돌(Conflict)이 심함.
 - **연관 매핑:** 주소 분할 없음. 전체가 [Tag]. 캐시 내 빈 방 아무 데나 유연하게 들어가 충돌이 없으나, 512개 전체 방을 동시 검색해야 하므로 비교기 하드웨어 단가가 매우 비쌘.
 - **세트 연관 매핑:** 주소를 [Tag | Set]으로 분할. 방을 묶어 세트(Set)를 만들고 세트 번호까지는 직접 매핑으로 찾아간 뒤, 세트 내부에서는 연관 매핑으로 방을 고르는 혼합 방식.
- **기록 방식 (Write Policy):**
 - **Write-through (즉시 기록):** 캐시와 RAM을 동시에 수정. 일관성이 완벽하나 속도가 느림.
 - **Write-back (나중 기록):** 캐시만 먼저 수정하고 해당 블록이 캐시에서 쫓겨날 때 RAM에 백업. 속도는 빠르나 일관성이 일시적으로 깨지므로 수정을 표시하는 **Dirty bit(더티 비트)** 관리가 필수적임.

12.6 가상 메모리 (Virtual Memory)

- **핵심 요약:** 보조기억장치의 공간을 주기억장치(RAM)의 확장 공간처럼 활용하여 사용자에게 거대한 메모리를 제공하는 마술 같은 기술입니다.
- **매핑 구조:** 가상 공간인 주소 공간은 **페이지(Page)** 단위로 쪼개고, 실제 물리 RAM 공간은 **블록(Block / Page Frame)** 단위로 쪼개어 매핑 테이블로 연결합니다.
- **페이지 결함 (Page Fault):** CPU가 요청한 가상 페이지가 진짜 RAM에 없을 때 발생하는 인터럽트 현상입니다. 디스크에서 페이지를 가져오는 동안 CPU는 다른 태스크를 실행합니다.
- **교체 알고리즘:** 가장 오래된 것을 교체하는 **FIFO**, 최근에 가장 사용되지 않은 것을 교체하는 **LRU (수명 레지스터 카운터 이용)**.

12.7 메모리 관리 하드웨어 (MMU)

- **핵심 요약:** 가상 주소를 물리 주소로 동적 변환하고 메모리 영역을 제어하는 특수 하드웨어 장치입니다.
- **핵심 기능 3가지:** 주소 변환(Translation), 메모리 보호(Protection), 메모리 공유(Sharing).
- **분할 방식 2가지:** 고정 크기로 쪼개어 관리하는 페이징(내부 단편화 발생 가능)과 의미 단위 가변 크기로 쪼개어 관리하는 세그멘테이션(외부 단편화 발생 가능).

■ [13장] 멀티프로세서 - 절별 조정밀 정리

13.1 멀티프로세서의 특징 (Characteristics)

- **핵심 요약:** 하나의 운영체제 통제 하에 공통의 메모리와 I/O 자원을 공유하며 병렬 처리를 수행하는 2개 이상의 CPU 탑재 시스템입니다.
- **구조적 결합도 분류 (★ 단골 출제):**
 - **공유 메모리 시스템 (Tightly Coupled - 강결합):** 모든 CPU가 하나의 공통 RAM 공간을 공유합니다. **프로그래밍이 매우 용이**하지만 메모리 접근 시 **병목 현상(접근 지연)** 발생 위험이 큼.
 - **분산 메모리 시스템 (Loosely Coupled - 약결합):** CPU마다 자기만의 독립된 로컬 메모리를 소유합니다. 메시지 패싱(Message-Passing)을 통해 통신합니다. Local 처리가 빨라 확장성이 좋으나 **프로그래밍 코드가 복잡하고 어렵**습니다.
- **클러스터 vs 그리드 컴퓨팅:** 고속 LAN으로 묶은 동종 컴퓨터 집합인 클러스터(Cluster)와 고속 WAN으로 대규모 원거리 이기종 컴퓨터들을 묶는 그리드(Grid).

13.2 상호연결 구조 (Interconnection Structures)

- **핵심 요약:** 여러 개의 CPU와 메모리 모듈들을 물리적으로 연결하는 하드웨어 토폴로지 구조입니다.
- **5가지 구조 특징 정리:**
 - **시분할 공통 버스:** 단일 공유 선에 다 붙임. 단순하고 저렴하나 장치가 많아지면 **통신 효율 대폭 감소(병목)**.
 - **다중 포트 메모리:** 메모리 칩 자체에 전용 접근 통로를 여러 개 배치. 전송 속도는 빠르나 **회선 비용이 엄청나게 증가**.
 - **크로스바 스위치:** 모든 CPU와 메모리 경로 사이에 $1:1$ 전용 교차점 스위치를 다 까는 방식. **속도는 최강이나 하드웨어 복잡도 및 비용이 가장 높음**.
 - **단단 교환망 (MIN):** 2입력 2출력 스위치 소자들을 층층이 단계별로 배치해 비용을 줄인 하이브리드 연결 구조 (예: 8x8 오메가 네트워크).
 - **하이퍼큐브 (Hypercube):** 차원에 따라 프로세서 수가 2^n 개로 기하급수적으로 확장되는 입체적 격자 연결 구조.

13.3 프로세서간 중재 (Arbitration)

- **핵심 요약:** 여러 CPU가 하나의 공통 버스를 사용하려고 동시 요청할 때 충돌하지 않도록 제어해 주는 알고리즘입니다.
- **동적 중재 알고리즘 5가지 명칭 (단답형 대비):**
 1. **시간 분할 (Time Slice):** 타임 슬롯을 라운드 로빈 방식으로 공평하게 배분.
 2. **폴링 (Polling):** 제어기가 순서대로 주소를 물어보며 중재.
 3. **LRU (Least Recently Used):** 가장 오랫동안 버스를 안 쓴 프로세서에게 최우선권 부여.

4. **FCFS (First-Come, First-Serve)**: 요청 순서대로 서비스 제공.

5. **회전 데이지 체인 (Rotating Daisy Chain)**: 권한 사용이 끝나면 우선순위를 맨 뒤로 회전시키는 방식.

13.4 프로세서간 통신과 동기화 (Communication & Synchronization)

- **핵심 요약**: 공유 자원을 여러 프로세서가 동시에 쓸 때 데이터 오염을 막기 위한 동기화 메커니즘입니다.
- **주요 용어 마스터**:
 - **상호배제 (Mutual Exclusion)**: 공용 자원을 한 번에 하나의 프로세서만 접근하도록 보장하는 기법입니다.
 - **임계 영역 (Critical Section)**: 공용 자원을 건드리므로 절대 동시에 두 개 이상의 CPU가 들어가면 안 되는 보안 프로그램 구역입니다.
 - **세마포 (Semaphore)**: 임계 영역에 프로세서가 있는지 없는지 체크해 주는 2진 상태 변수 (0 또는 1)입니다.
 - **Test and Set 명령어**: 세마포어 값을 읽어오는 작업과 1로 세팅하는 작업을 하드웨어적으로 원자적(Atomic, 단일 단계)으로 완벽히 한 번에 수행하여 동기화를 보장하는 특수 하드웨어 명령어입니다.

13.5 캐시 일관성 (Cache Coherence)

- **핵심 요약**: 여러 CPU가 동일한 RAM 소스 데이터를 각자의 로컬 캐시에 복사해 간 상태에서, 한쪽이 데이터를 수정했을 때 다른 쪽 캐시들의 데이터가 쓰레기 값이 되는 불일치 문제입니다. (★ 서술형 단골)
- **해결 프로토콜 2가지 원리**:
 1. **디렉토리 방법**: 중앙집중식 디렉토리 표를 메모리에 두고 어떤 블록이 어디로 복사되어 가 있는지 상태(읽기전용 RO / 읽기쓰기 RW)를 전역적으로 추적 및 통제하는 방식입니다.
 2. **스누핑 방법 (Snooping)**: 공유 버스를 사용하는 시스템에서 각 캐시 제어기가 버스의 쓰기 신호를 실시간으로 감시(Snooping)하고 있다가, 타 CPU가 내 구역 데이터를 수정하는 순간 내 캐시의 **Valid bit**를 **무효화(0으로 제어)** 시켜 일관성을 유지하는 기술입니다.